The Journal of Open Source Software

Pyrimidine: An algebra-inspired Programming framework for evolutionary algorithms

Congwei Song [1]

1 Beijing Institute of Mathematical Sciences and Applications, Beijing, China

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Sébastien Boisgérault

Reviewers:

- @mmore500
- @sjvrijn

Submitted: 11 December 2023
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Pyrimidine: An algebra-inspired Programming framework for evolutionary algorithms

Summary

Pyrimidine stands as a versatile framework designed for GAs, offering exceptional extensibility for a wide array of evolutionary algorithms, including particle swarm optimization and difference evolution.

Leveraging the principles of object-oriented programming (OOP) and the meta-programming, we introduce a distinctive design paradigm is coined as "algebra-inspired Programming" signifying the fusion of algebraic methodologies with the software architecture.

Statement of need

As one of the earliest developed optimization algorithms (Holland, 1975; Katoch et al., 2021), the genetic algorithm (GA) has found extensive application across various domains and has undergone modifications and integrations with new algorithms (Alam et al., 2020; Cheng & Alkhalifah, 2023; Katoch et al., 2021). The principles of GA will not be reviewed in this article. For a detailed understanding, please refer to references (Holland, 1975; Simon, 2013) and the associated literatures.

In a typical Python implementation, populations are initially defined as lists of individuals, with each individual represented by a chromosome composed of a list of genes. Creating an individual can be achieved utilizing either the standard library's `array` or the widely-used third-party library `numpy` (NumPy Developers, 2022). Following this, evolutionary operators are defined and applied to these structures.

A concise comparison between `pyrimidine` and several popular frameworks is provided in Table 1, such as `DEAP` (Fortin et al., 2012) and `gaft`, which have significantly influenced the design of `pyrimidine`.

Table 1: Comparison of the popular genetic algorithm frameworks.

Library	Design Style	Versatility	Extensibility	Visualization
`pyrimidine`	OOP, Meta-programming, Algebra-insprited	Universal	Extensible	export the data in `DataFrame`

Library	Design Style	Versatility	Extensibility	Visualization
DEAP	OOP, Functional, Meta-programming	Universal	Limited by its philosophy	export the data in the class LogBook
gaft	OOP, decoration pattern	Universal	Extensible	Easy to Implement
geppy	based on DEAP	Symbolic Regression	Limited	-
tpot (Olson et al., 2016)/gama (Gijsbers & Vanschoren, 2021)	scikit-learn Style	Hyperparameter Optimization	Limited	-
gplearn/pysr	scikit-learn Style	Symbolic Regression	Limited	-
scikit-opt	scikit-learn Style	Numerical Optimization	Unextensible	Encapsulated as a data frame
scikit-optimize	scikit-learn Style	Numerical Optimization	Very Limited	provide some plotting function
NEAT (McIntyre et al., n.d.)	OOP	Neuroevolution	Limited	use the visualization tools

Tpot/gama, gplearn/pysr, and scikit-opt follow the scikit-learn style (Buitinck et al., 2013), providing fixed APIs with limited extensibility. They are merely serving their respective fields effectively (as well as NEAT).

DEAP is feature-rich and mature. However, it primarily adopts a tedious meta-programming style. Some parts of the source code lack sufficient decoupling, limiting its extensibility. Gaft is a highly object-oriented software with excellent scalability, but it is currently inactive.

Pyrimidine fully utilizes the OOP and meta-programming capabilities of Python, making the design of the APIs and the extension of the program more natural. So far, We have implemented a variety of intelligent algorithms by pyrimidine, including adaptive GA (Hinterding et al., 1997), quantum GA (Supasil et al., 2021), differential evolution (Radtke et al., 2020), evolutionary programming, particle swarm optimization (Wang et al., 2018), as well as some local search algorithms, such as simulated annealing.

This library provides a wide range of chromosome classes in the chromosome module, including Boolean, integer, and real number types, and that can even represent probability distributions and node permutations in graph and their mixed forms. Most of them are subclasses of numpy.ndarray, the array class of numpy, but custom definitions are also allowed. Each class implements corresponding genetic operations such as crossover and others.

In the benchmarks module, we offer a comprehensive array of problems to evaluate various algorithms, encompassing both traditional optimization models and cutting-edge machine learning models.

Algebra-inspired programming

The innovative approach is termed "algebra-inspired Programming." It should not be confused with so-called algebraic programming, but it draws inspiration from its underlying principles.

The advantages of the model are summarized as follows:

1. The population system and genetic operations are treated as an algebraic system, and genetic algorithms are constructed by imitating algebraic operations.
2. It has better scalability. For example it is easy to define multi-populations, as a high-order algebraic system.
3. The code is more concise.

Basic concepts

We introduce the concept of a **container**, simulating an **(algebraic) system** where specific operators are not yet defined.

A container s of type S, with elements of type A, is represented by following expression:

$$s = \{a : A\} : S \quad \text{or} \quad s : S[A] \tag{1}$$

where the symbol $\{\cdot\}$ signifies either a set, or a sequence to emphasize the order of the elements. The notation $S[A]$ mimicks Python syntax, borrowed from the module typing.

Building upon the foundational concept, we define a population in `pyrimidine` as a container of individuals. The introduction of multi-population further extends this notion, representing a container of populations, often referred to as "the high-order container". `Pyrimidine` distinguishes itself with its inherent ability to seamlessly implement multi-population GAs. Populations in a multi-population behave analogously to individuals in a population. Notably, it allows to define containers in higher order, such as a container of multi-populations, potentially intertwined with conventional populations.

While an individual can be conceptualized as a container of chromosomes, it will not necessarily be considered a system. Similarly, a chromosome might be viewed as a container of genes (implemented by the arrays in practice).

In a population system s, the formal representation of the crossover operation between two individuals is denoted as $a \times_s b$, that can be implemented as the command `s.cross(a, b)`. Although this system concept aligns with algebraic systems, the current version of our framework diverges from this notion, and the operators are directly defined as methods of the elements, such as `a.cross(b)`.

The lifting of a function/method f is a common approach to defining the function/method for the system:

$$f(s) := \{f(a)\}$$

unless explicitly redefined. For example, the mutation of a population typically involves the mutation of all individuals in it, but there are cases where it may be defined as the mutation of a randomly selected individual. Another type of lifting is that the fitness of a population is determined as the maximum of the fitness values among the individuals in the population.

`transition` is the primary method in the iterative algorithms, denoted as a transform:

$$T(s) : S \to S$$

The iterative algorithms can be represented as $T^n(s)$.

Metaclasses

The metaclass `System` is defined to simulate abstract algebraic systems, which are instantiated as a set containing a set of elements, as well as operators and functions on them.

`Container` is the super-metaclass of `System` for creating containers.

DRAFT

Mixin classes

Mixin classes specify the basic functionality of the algorithm.

The `FitnessMixin` class is dedicated to the iteration process focused on maximizing fitness, and its subclass `PopulationMixin` represents the collective form.

When designing a novel algorithm, significantly differing from the GA, it is advisable to start by inheriting from the mixin classes and redefining the `transition` method, though it is not mandatory.

Base Classes

There are three base classes in `pyrimidine`: `BaseChromosome`, `BaseIndividual`, `BasePopulation`, to create chromosomes, individuals and populations respectively.

For convenience, `pyrimidine` provides some commonly used subclasses, where the genetic operations are implemented such as, `cross` and `mutate`. Especially, `pyrimidine` offers `BinaryChromosome` for the binary encoding as used in the classical GA.

Generally, the algorithm design starts as follows, where `MonoIndividual`, a subclass of `BaseIndividual`, just enforces that the individuals can only have one chromosome.

```
class UserIndividual(MonoIndividual):
    element_class = BinaryChromosome
    # default_size = 1

    def _fitness(self):
        # Compute the fitness

class UserPopulation(StandardPopulation):
    element_class = UserIndividual
    default_size = 10
```

In the codes, `UserIndividual` (respectively `UserPopulation`) is a container of elements in type of `BinaryChromosome` (respectively `UserIndividual`). Following is the equivalent expression, using the notion in Equation 1 :

```
UserIndividual = MonoIndividual[BinaryChromosome]
UserPopulation = StandardPopulation[UserIndividual] // 10
```

Notably, users are not encouraged to directly override the `fitness` attribute, but instead to override the `_fitness` method, where users should define the specific fitness calculation process. The operator `// 10` creates a population with 10 individuals by default.

The code can be further simplified. Algebraically, there is no difference between `MonoIndividual`, the individual class with a single chromosome, and `Chromosome`. And the population also can be treated as a container of chromosomes. See the following codes.

```
class UserChromosome(BaseChromosome):
    def _fitness(self):
        # Compute the fitness

# population as a container of chromosomes,
# instead of individuals
UserPopulation = StandardPopulation[UserChromosome] // 10
```

DRAFT

An example to begin

In this section, we demonstrate the basic usage of `pyrimidine` with the classic 0-1 knapsack problem, whose solution can be naturally encoded in binary format without the need for additional decoding:

$$\max \sum_i c_i x_i \text{st} \sum_i w_i x_i \leq W, \quad x_i = 0, 1, i = 1, \cdots, n$$

where c_i and w_i represents the value and the weight of the i-th bag respectively, and x_i is a binary variable indicating whether the i-th bag is taken or not.

```
from pyrimidine import BinaryChromosome, MonoIndividual, StandardPopulation
from pyrimidine.benchmarks.optimization import Knapsack

n = 50
_evaluate = Knapsack.random(n)  # the objective function

class UserIndividual(MonoIndividual):
    element_class = BinaryChromosome // n
    def _fitness(self):
        return _evaluate(self[0])

"""
equivalent to:
UserIndividual = MonoIndividual[BinaryChromosome // n]
.set_fitness(lambda o: _evaluate(o[0]))
"""

UserPopulation = StandardPopulation[UserIndividual] // 20
```

Using chromosome as the population's elements, we arrange all the components in a single line:

```
UserPopulation = StandardPopulation[BinaryChromosome // n].set_fitness(_evaluate)
```

Then we execute the evolutionary program as follows.

```
pop = UserPopulation.random()
pop.evolve(max_iter=100)
```

Finally, the optimal individual can be obtained with `pop.best_individual`, or `pop.solution` to decode the individual to the solution of the problem.

Visualization

Instead of implementing visualization methods, `pyrimidine` yields a `pandas.DataFrame` object that encapsulates statistical results for each generation by setting `history=True` in the `evolve` method. Users can harness this object to plot the performance curves. Generally, users are required to furnish a "statistic dictionary" whose keys are the names of the statistics, and values are functions mapping the population to numerical values, or strings presenting pre-defined methods or attributes of the population.

```
# statistic dictionary, computing the mean, the maximum and
# the standard deviation of the fitnesses for each generation
stat = {'Mean Fitness': 'mean_fitness',
'Best Fitness': 'max_fitness',
```

DRAFT

```
'Standard Deviation of Fitnesses': lambda pop: np.std(pop.get_all_fitness())
}

# obtain the statistical results through the evolution.
data = pop.evolve(stat=stat, max_iter=100, history=True)
```

data is an pandas.DataFrame object, with the columns "Mean Fitness", "Best Fitness" and "Standard Deviation of Fitnesses". Now utilize the plot method of the object (or the Python library matplotlib) to show the iteration history Figure 1.

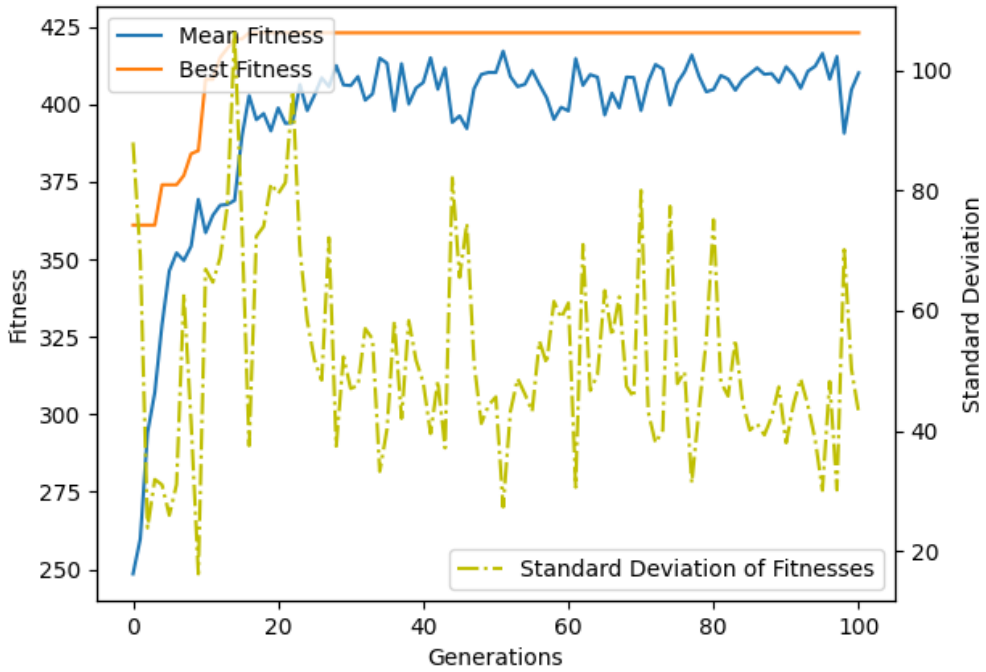

Figure 1: The fitness evolution curve of the population.

You can also set verbose=True in the evolve method to see each step of the iteration. If you do not want to set anything, then it is recommended to use the ezolve method, such as pop.ezolve().

Conclusion

I have conducted extensive experiments and improvements, showcasing that pyrimidine is a versatile framework suitable for implementing various evolution algorithms. Its design offers strong extensibility, allowing the implementation of any iterative algorithm, such as simulated annealing or particle swarm optimization. For users developing new algorithms, pyrimidine is a promising choice.

We have not implemented parallel computation yet which is important for intelligent algorithms, but we have set up an interface that can be utilized at any time. The full realization of algebraic programming concepts is still in progress. The functionality of symbolic regression has not been realized yet, but we are considering reusing what DEAP provides rather than reinventing the wheel. Certainly, there is ample room for further improvement.

References

Alam, T., Qamar, S., Dixit, A., & Benaida, M. (2020). Genetic algorithm: Reviews, implementations, and applications. *CompSciRN: Computer Principles (Topic)*. https://api.semanticscholar.org/CorpusID:219914744

Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., Niculae, V., Prettenhofer, P., Gramfort, A., Grobler, J., Layton, R., VanderPlas, J., Joly, A., Holt, B., & Varoquaux, G. (2013). API design for machine learning software: Experiences from the scikit-learn project. *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 108–122.

Cheng, S., & Alkhalifah, T. (2023). Robust data driven discovery of a seismic wave equation. *Geophysical Journal International*, *236*(1), 537–546. https://doi.org/10.1093/gji/ggad446

Fortin, F.-A., Rainville, F.-M. D., Gardner, M.-A., Parizeau, M., & Gagné, C. (2012). DEAP: Evolutionary algorithms made easy. In *Journal of Machine Learning Research* (Vol. 13, pp. 2171–2175).

Gijsbers, P., & Vanschoren, J. (2021). GAMA: A general automated machine learning assistant. In Y. Dong, G. Ifrim, D. Mladenić, C. Saunders, & S. Van Hoecke (Eds.), *Machine learning and knowledge discovery in databases. Applied data science and demo track* (pp. 560–564). Springer International Publishing.

Hinterding, R., Michalewicz, Z., & Eiben, A. E. (1997). Adaptation in evolutionary computation: A survey. *Proceedings of 1997 IEEE International Conference on Evolutionary Computation (ICEC '97)*, 65–69. https://doi.org/10.1109/ICEC.1997.592270

Holland, J. (1975). *Adaptation in natural and artificial systems*. The Univ. of Michigan.

Katoch, S., Chauhan, S. S., & Kumar, V. (2021). A review on genetic algorithm: Past, present, and future. In *Multimed Tools Appl* (Vol. 80, pp. 8091–8126). https://doi.org/10.1007/s11042-020-10139-6

McIntyre, A., Kallada, M., Miguel, C. G., Feher de Silva, C., & Netto, M. L. (n.d.). *Neat-python*.

NumPy Developers. (2022). *Numpy - the fundamental package for scientific computing with python*. NumPy Developers. https://numpy.org/

Olson, R. S., Urbanowicz, R. J., Andrews, P. C., Lavender, N. A., Kidd, L. C., & Moore, J. H. (2016). Automating biomedical data science through tree-based pipeline optimization. In *Journal of Machine Learning Research* (pp. 123–137). https://doi.org/10.1007/978-3-319-31204-0_9

Radtke, J. J., Bertoldo, G., & Marchi, C. H. (2020). DEPP - differential evolution parallel program. *Journal of Open Source Software*, *5*(47), 1701. https://doi.org/10.21105/joss.01701

Simon, D. (2013). *Evolutionary optimization algorithms: Biologically inspired and population-based approaches to computer intelligence*. John Wiley & Sons.

Supasil, J., Pathumsoot, P., & Suwanna, S. (2021). Simulation of implementable quantum-assisted genetic algorithm. *Journal of Physics: Conference Series*, *1719*(1), 012102. https://doi.org/10.1088/1742-6596/1719/1/012102

Wang, D., Tan, D., & Liu, L. (2018). Particle swarm optimization algorithm: An overview. *Soft Computing*, *22*(2), 387–408. https://doi.org/10.1007/s00500-016-2474-6

DRAFT